

Мета роботи

Ознайомитись із методами реалізації багатозадачності у вбудованих системах на базі мікроконтролера PIC18F4550, а також отримати практичні навички роботи з таймерами, перериваннями, пріоритетами переривань і розподілом процесорного часу між кількома задачами.

Короткий план виконання роботи

1. Створено проєкт для мікроконтролера PIC18F4550 у середовищі MPLAB C18 з урахуванням роботи через bootloader.
2. Реалізовано шаблон з переадресацією векторів переривань та налаштовано пріоритетні переривання.
3. Ініціалізовано периферійні модулі TMR0, USART і CCP1.
4. У спільному проєкті реалізовано навчальні приклади з багатозадачності.
5. Перевірено роботу обробників переривань, реакцію на натискання кнопок та формування ШІМ-сигналу.
6. Виконано тестування проєкту в симуляторі.

Код програми

```
1 #include <pl8f4550.h>
2 #include <delays.h>
3
4 #define K1 0x01
5 #define K2 0x02
6 #define K3 0x04
7 #define PIN_Keys PORTE
8 #define PIN_K1 PORTB
9 #define PIN_K2 PORTB
10 #define PIN_K3 PORTB
11 #define Keypad (K1 | K2 | K3)
12
13
14 unsigned int counter;
15 unsigned int keycntr;
16 struct flagsStruct
17 {
18     unsigned int clicking;
19     unsigned int pushing;
20 };
21 struct flagsStruct flags;
22 unsigned char press_code;
23 unsigned int adc_value;
24 char buffer[6];
25 unsigned char k;
26
27 void systemInit(void);
28 void TMR0Init(void);
29 void USARTInit(void);
30 void ADCInit(void);
31 void PWMInit(void);
32
33 void PWMInit(void);
34
35 void TMR0Interrupt(void);
36
37 void sendCharUSART(char);
38 void sendWordUSART(char*);
39 void numberToWord(unsigned int, char*);
40
41 unsigned char keyboard(void);
42 void PWMSetup(unsigned char);
43
44 extern void _startup (void);
45 void ISRHigh(void);
46
47 #pragma code main=0x102A
48 void main(void)
49 {
50     TRISB = 0;
51     ADCON1 = 0x0F;
52     counter = 1;
53     keycntr = 0;
54     adc_value = 0;
55
56     systemInit();
57     TMR0Init();
58     USARTInit();
59     ADCInit();
60     PWMInit();
61
62     while (1)
63     {
```

```
61 {
62     sendCharUSART('\f');
63     if (press_code & 128) sendCharUSART('L');
64     if (press_code)
65     {
66         unsigned char multiplier;
67
68         sendCharUSART('B');
69         sendCharUSART((press_code & 7) | '0');
70         multiplier = 1 << ((press_code & 7) - 1);
71         multiplier *= (press_code & 128) ? (8) : 1;
72         sendCharUSART('M');
73         numberToWord(multiplier, buffer);
74         sendWordUSART(buffer);
75         PWMSetup(multiplier);
76     }
77
78     for (k = 0; k < 1; k++)
79     {
80         Delay10KTCYx(250);
81     }
82 }
83
84
85 #pragma code interrupt_high_vector=0x1008
86 void interrupt_high_vector(void)
87 {
88     __asm goto ISRHigh __endasm
89 }
90
91
92 #pragma code
93
94 #pragma interrupt ISRHigh
95 void ISRHigh(void)
96 {
97     if (INTCONbits.TMR0IF)
98     {
99         INTCONbits.TMR0IF = 0;
100         TMR0Interrupt();
101     }
102     // Delay10KTCYx(250);
103 }
104
105 void systemInit(void)
106 {
107     INTCONbits.GIE = 1;
108     INTCONbits.PEIE = 1;
109 }
110
111 void TMR0Init(void)
112 {
113     T0CON = 0b10010001;
114     INTCONbits.TMR0IE = 1;
115     INTCON2bits.TMR0IP = 1;
116 }
117
118 void USARTInit(void)
119 {
120     SPBRGH = 0;
```

```

120 {
121     SPBRGH = 0;
122     BAUDCONbits.BRG16 = 0;
123
124     SPBRG = 77;
125     TXSTAbits.BRGH = 0;
126     TXSTAbits.SYNC = 0;
127     RCSTAbits.SPEN = 1;
128
129     TXSTAbits.TX9 = 0;
130     TXSTAbits.TXEN = 1;
131 }
132
133 void ADCInit(void)
134 {
135     ADCON0 = 0b00000001;
136     ADCON1 = 0b00001110;
137     ADCON2 = 0b00101110;
138
139     ADCON0bits.GO = 1;
140 }
141
142 void PWMInit(void)
143 {
144     CCP1CON = 0b00001100;
145     T2CON = 0b00000000;
146     TRISBbits.RC1 = 0;
147     TRISBbits.RC2 = 0;
148
149     TRISBbits.RB3 = 0;
150 }
151
152 void TMR0Interrupt(void)
153 {
154     unsigned char current_press_code = keyboard();
155     if (current_press_code) press_code = current_press_code;
156 }
157
158 void PWMSetup(unsigned char multiplier)
159 {
160     if (multiplier)
161     {
162         T2CONbits.TMR2ON = 1;
163         if (multiplier != 32)
164         {
165             T2CONbits.T2CKPS = 2;
166             PR2 = (240 / multiplier) - 1;
167         }
168         else if (multiplier == 32)
169         {
170             T2CONbits.T2CKPS = 1;
171             PR2 = 29;
172         }
173         CCP1L = (PR2 + 1) / 2;
174     }
175     else
176     {
177         T2CONbits.TMR2ON = 0;
178     }
179 }
180
181 void sendCharUSART(char data)

```

```

180
181 void sendCharUSART(char data)
182 {
183     while(TXSTAbits.TRMT == 0);
184     TXREG = data;
185 }
186
187 void sendWordUSART(char* word)
188 {
189     int i = 0;
190     // sendCharUSART('\f');
191     while(word[i] != '\0' && i != 5)
192     {
193         sendCharUSART(word[i]);
194         i++;
195     }
196     // sendCharUSART('\x');
197 }
198
199 void numberToWord(unsigned int number, char* word)
200 {
201     register unsigned char n;
202     for(n = 0; n <= 4; n++)
203     {
204         word[n] = '0';
205     }
206     for(n = 0; n <= 4; n++)
207     {
208         word[4-n] = (number % 10) | '0';
209         number /= 10;
210         if (!number) break;
211     }
212 }

```

```

210         if (!number) break;
211     }
212 }
213
214 unsigned char keyboard(void)
215 {
216     struct flags
217     {
218         unsigned int clicking;
219         unsigned int pushing;
220     };
221     unsigned char instant = (PIN_Keys & Keypad) & 0b00000111;
222
223     if (instant != Keypad)
224     {
225         int i;
226         unsigned char button_counter = 0;
227         for (i = K1; i <= K3; i+=2)
228         {
229             button_counter++;
230             if (!(instant & i))
231             {
232                 if (!flags.clicking)
233                 {
234                     flags.clicking = 1;
235                     return button_counter;
236                 }
237             }
238             if ((!flags.pushing) && (keycntr > 12))
239             {
240                 flags.pushing = 1;
241             }
242         }
243     }
244 }

```

```

240         flags.pushing = 1;
241         return button_counter | 0x80;
242     }
243     keycntr++;
244 }
245 }
246 }
247 else
248 {
249     flags.pushing = 0;
250     flags.clicking = 0;
251     keycntr = 0;
252 }
253 return 0;
254 }

```

Результат виконання у середовищі Proteus (детальніше у демонстрації):

Звичайне натиснення кнопки 1:

